



CFA - Priorizah

Apresentação Técnica

Detalhes técnicos do projeto, construção física e recursos utilizados



Índice

01

Construção Física

Módulos utilizados,
configurações e estrutura

02

Lógica de Funcionamento

Esquema de funcionamento do
projeto e lógica

03

Software e Bibliotecas

Estrutura do código e recursos
de software utilizados

04

Resultados e Limitações

O que aprendemos, o que não
consequimos fazer e próximos passos



01

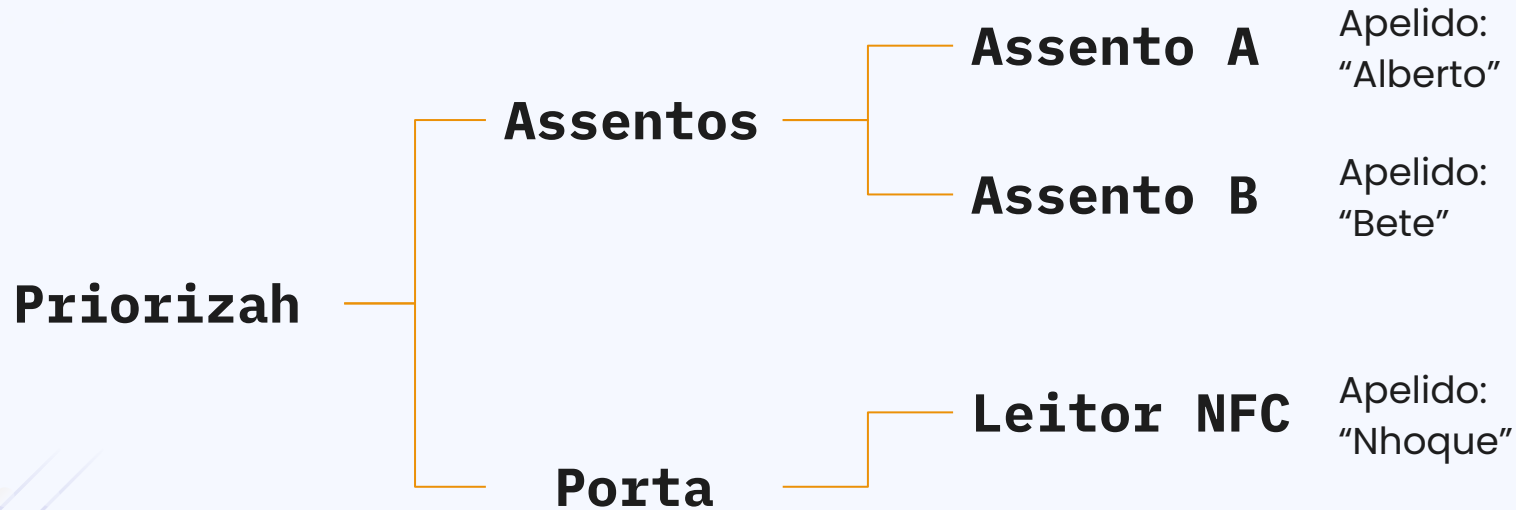
Construção Física



1.1. Módulos Utilizados

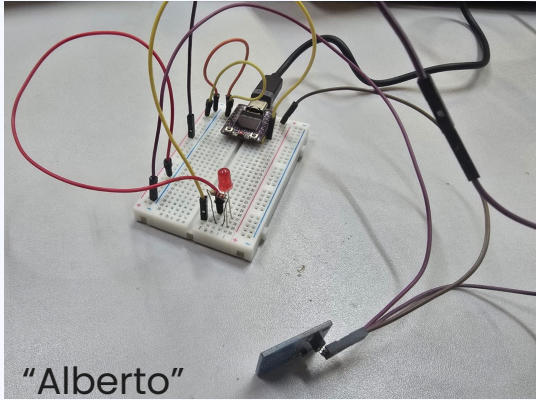
3 ESP32-C3 mini	Microcontrolador para realizarmos a execução do sistema e controle dos componentes.
1 PN532	Componente leitor de cartões/tags NFC.
1 Buzzer	Componente para emissão de sinais sonoros, usados aqui como “aviso”.
4 TTP223B	Módulo sensor de toque capacitivo, usado nos assentos.
2 LED	Componente de luz, aqui usado para indicar o estado do sistema de assentos.
2 Protoboard	Plataforma de suporte que permite montar o circuito sem solda.
N Jumper	Fios de conexões entre os componentes.
Servidor	PC ou outro dispositivo para atuar como servidor.

1.2. Estrutura do projeto



OBS: Arquitetura distribuída composta por um módulo leitor NFC, dois módulos de assento e um servidor central responsável pela coordenação.

1.2.1. “Alberto” e “Bete”

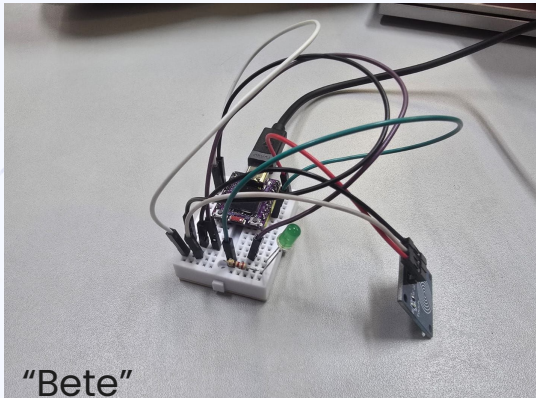


Componentes por sistema

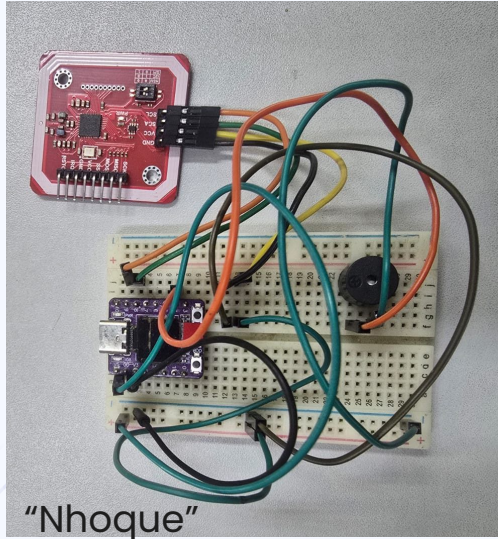
- 2 sensores de toque capacitivo (TTP223B);
- 1 LED indicador;
- 1 ESP32-C3 Mini.

Funcionamento geral

- Os dois sensores detectam se o assento está ocupado (usando um algoritmo de consenso para atualizar o status);
- LED indica visualmente se o assento está livre ou ocupado;
- ESP32 envia o estado do assento ao servidor via TCP/IP.



1.2.2. “Nhoque”



Componentes do sistema

- 1 módulo leitor NFC (PN532)
- 1 ESP32-C3 Mini
- 1 Buzzer

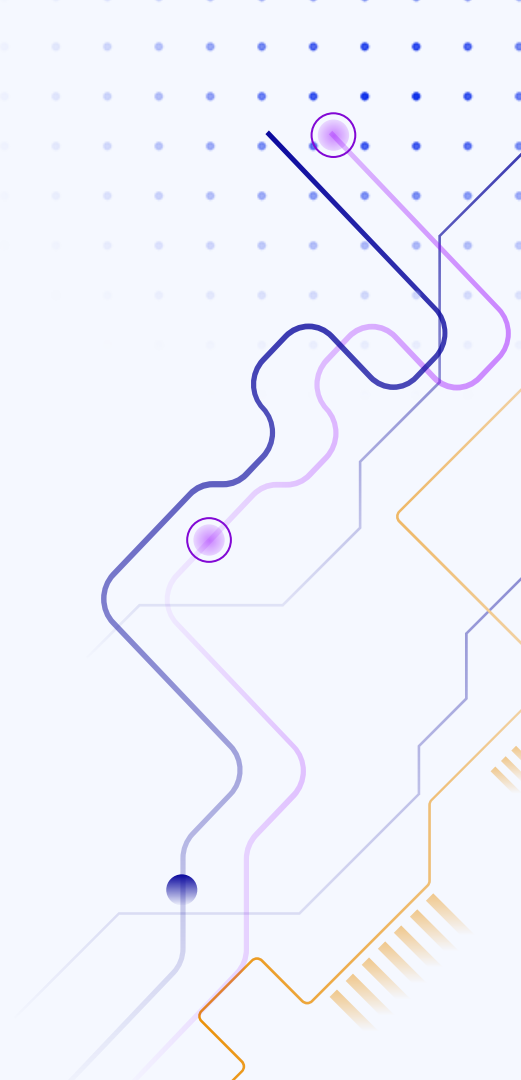
Funcionamento geral

- Usuário aproxima o cartão/chaveiro NFC;
- O PN532 identifica o passageiro preferencial;
- O ESP32 envia a informação ao servidor via TCP/IP;
- O buzzer confirma a leitura e o sistema aciona os avisos.



02

Lógica e Funcionamento



2.1. Lógica de Ativação



2.2. Ordem de eventos

Uma **pessoa com necessidade de assento preferencial** passa seu cartão no leitor, próximo a porta

Primeiro



O ESP32 do anexoado ao leitor NFC **valida se o cartão lido é válido** para a prioridade

Segundo



O ESP32 manda um sinal ao servidor **confirmando a leitura válida**

Terceiro



Quarto

O servidor manda uma mensagem a todos os ESP32s de assentos para consultar sua disponibilidade

Quinto

Os ESP32s dos assentos retornam qual a disponibilidade de cada assento

Sexto

Caso algum assento esteja livre, sua luz acenderá. Caso contrário, o servidor receberá a negativa de todos os assentos, e mandará uma mensagem ao ESP32 da porta para tocar o Buzzer

2.3. Lógica de Ocupação dos Assentos

Para detectar se um assento foi ocupado, utilizamos 2 sensores de toque, com as seguintes propriedades:

- Leitura a cada 0.5s
- Atualização de estado a cada 5s
- Consenso: caso qualquer um dos dois sensores detecte um toque, será considerado que o assento está ocupado, usando a lógica OR (durante o período de 5s)

Exemplo:

Sensor 1

0
0
0
0
0
0
0
1
1
1
0
0
0
1
1

t
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
5.5
6.0
6.5
7.0

Sensor 2

1
1
1
1
0
0
0
1
1
1
1
0
0
0
0

=1
=1
=1
=1
=0
=0
=1
=1
=1
=1
=1
=0
=1
=1

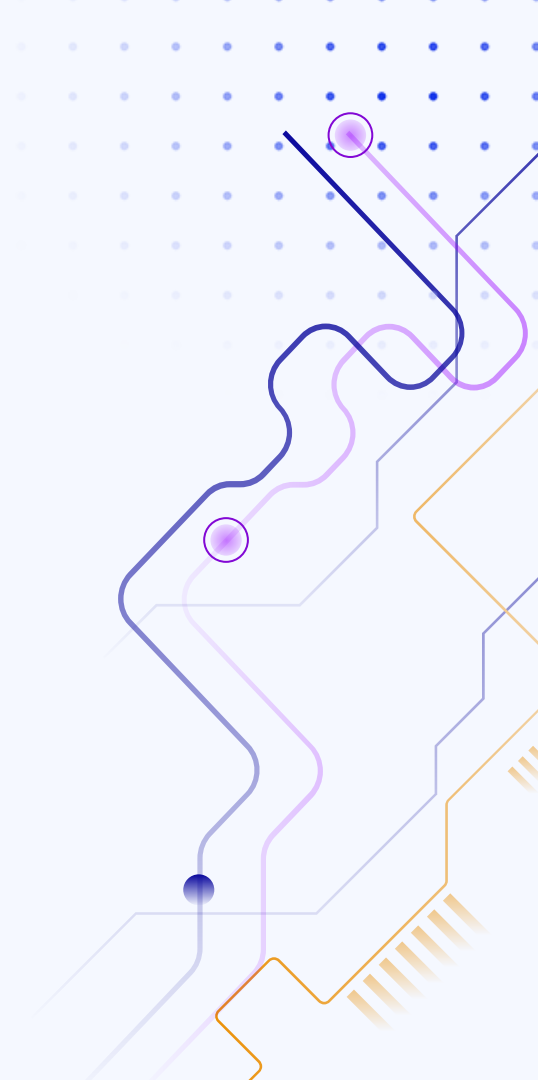
1

1



03

Software e Bibliotecas



3.1. Estrutura do Código

- Utilizando MicroPython, obtivemos 3 blocos principais com finalidades e funções bem definidas, divididas em pastas de código:
 - a. Módulo de leitura NFC: localizado em 'esp_comunicando.py' e 'nfc_buzzer.py' (leitor PN532 e buzzer), que também possui uma camada de inicialização em 'start.py', que chama o fluxo principal do leitor.
 - b. Módulo de assentos: implementado em 'sensor_vl.py' (sensores de toque capacitivo TTP223 e led) e 'seat_state.py' (lógica de estado).
 - c. Servidor central: implementado em 'pc_server.py'. Comunicação baseada em: receber mensagens dos ESPs, manter um registro dos assentos conectados, consultar o estado deles e decidir qual assento deve receber o sinal de indicação após um evento NFC.

OBS: há arquivos de configuração local, como `esp_config.py`, que definem Wi-Fi, IP do servidor e parâmetros específicos de cada dispositivo.

3.1.1 Responsabilidade de cada ESP32

ESP32 do Leitor de NFC:

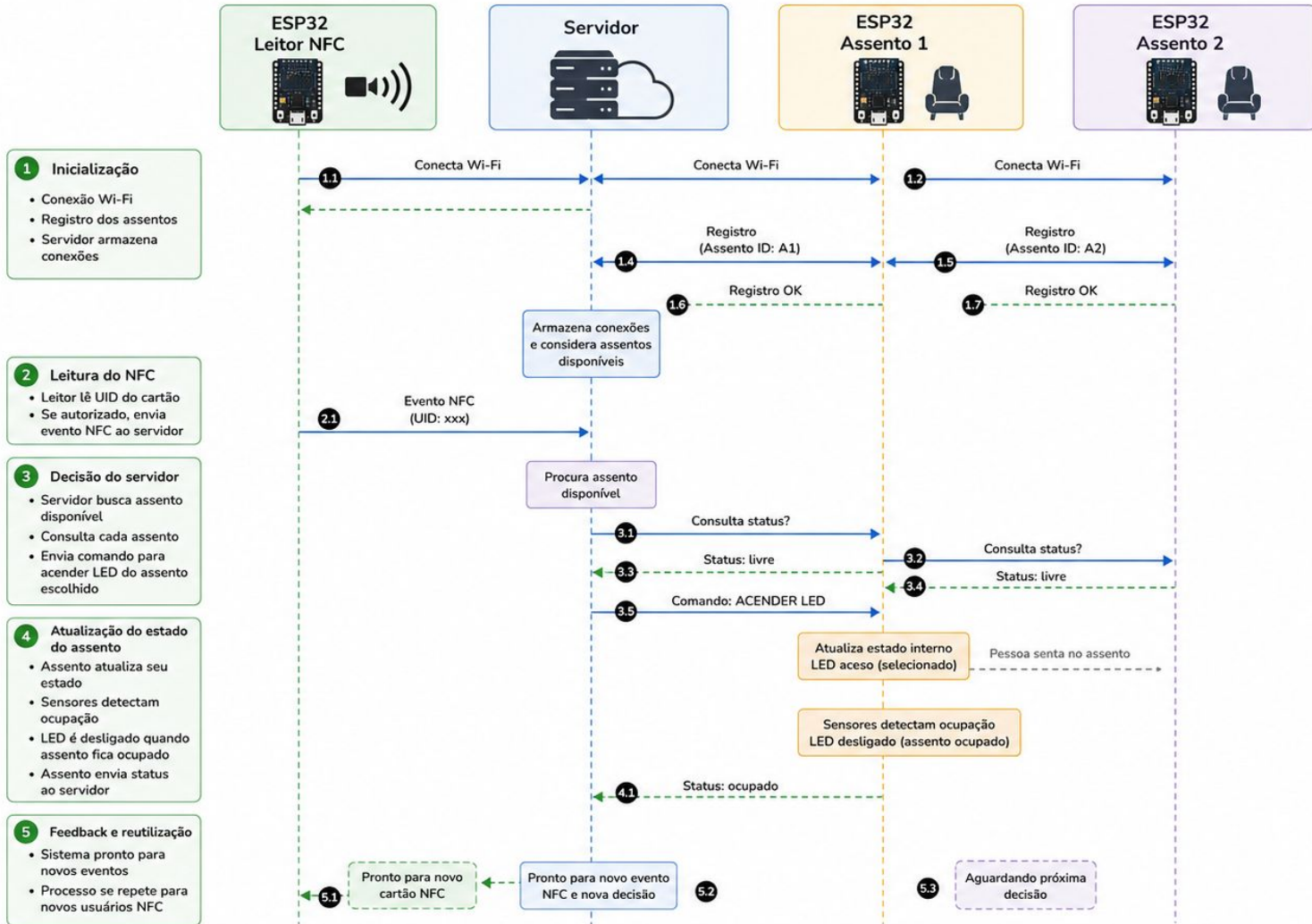
- Detecta a presença de um cartão/chaveiro próximo ao leitor PN532;
- Verifica se o UID está na lista de identificadores autorizados;
- Envia uma mensagem para o servidor;
- Aciona um buzzer para fornecer feedback sonoro ao usuário.

ESP32 do Assentos:

- Monitora dois sensores de toque capacitivo;
- Verifica se alguém está sentado;
- Mantém um estado interno do assento;
- Controla um LED que indica o status do assent;
- Responde a comandos vindos do servidor (ex: status do assento).

3.1.2 Fluxo de Comunicação





3.1.2.1 Tabela de Mensagens

Mensagem	Origem	Destino	Descrição
seat_register	ESP de Assento	Servidor	Registra um assento no sistema.
get_status	Servidor	ESP de Assento	Solicita o estado atual do assento.
seat_status	ESP de Assento	Servidor	Retorna o estado do assento.
set_led	Servidor	ESP de Assento	Comanda ligar ou desligar o LED.
set_led_result	ESP de Assento	Servidor	Confirma a execução do comando do LED.
nfc_*	ESP de NFC	Servidor	Envia eventos relacionados à leitura de cartões NFC.

3.2. Bibliotecas Utilizadas

- Machine: usada em código para MicroPython no ESP32. Ela permite acessar recursos de hardware diretamente, exemplo no projeto: configurar o leitor NFC PN532, controlar o buzzer e ler os sensores de toque e acionar o LED.
- Network: usada para gerenciar a conexão Wi-Fi no ESP32 para comunicação.
- Socket: usada para criar conexões de rede TCP/IP a fim de compartilhar dados.
- Uasyncio: executar tarefas simultaneamente.
- Json: transformar dicionários Python em texto e vice-versa para nossas mensagens estruturadas.
- Time: usada para controle temporal.
- Micropython.const: usada para definir constantes de forma eficiente no MicroPython. Ela ajuda a criar valores simbólicos para comandos do PN532 e outras configurações fixas.
- Threading: usada no servidor para lidar com várias conexões simultaneamente.

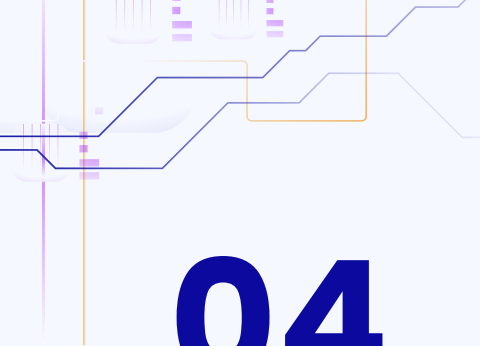
3.2. Bibliotecas Utilizadas

- Machine → Controle do hardware (GPIO, LED, sensores e buzzer)
- Network → Conexão Wi-Fi
- Socket → Comunicação TCP/IP
- JSON → Estrutura das mensagens
- uasyncio → Execução concorrente
- Threading → Múltiplas conexões no servidor
- Time → Temporização
- micropython.const → Constantes do protocolo PN532

3.3. Outros recursos de Software

- Gerenciamento de estado dos assentos (ocupado/livre e controle do LED).
- Programação assíncrona (uasyncio) para executar múltiplas tarefas no ESP32.
- Concorrência (threading) no servidor para atender vários dispositivos simultaneamente.
- Tratamento de erros e reconexão automática em falhas de comunicação.
- Arquivos de configuração para Wi-Fi, IP do servidor e identificação dos dispositivos.
- Testes automatizados para validar registro de assentos, comandos, timeouts e mensagens.

Assim, aumenta-se a confiabilidade, escalabilidade e facilidade de manutenção do sistema.



04

Resultados e Limitações



4.1. O que aprendemos?

NFC:

- Funcionamento do módulo NFC
- Como a leitura de NFC funciona (distância, informações, disponibilidade de leitura, etc)

Sensor de Toque

- Como a leitura do sensor funciona (distância, informações, limitações, etc)

ESP32:

- Como estabelecer uma conexão estável entre um PC e um ESP32 com TCP/IP
- Como conectar e ler as saídas dos módulos utilizados

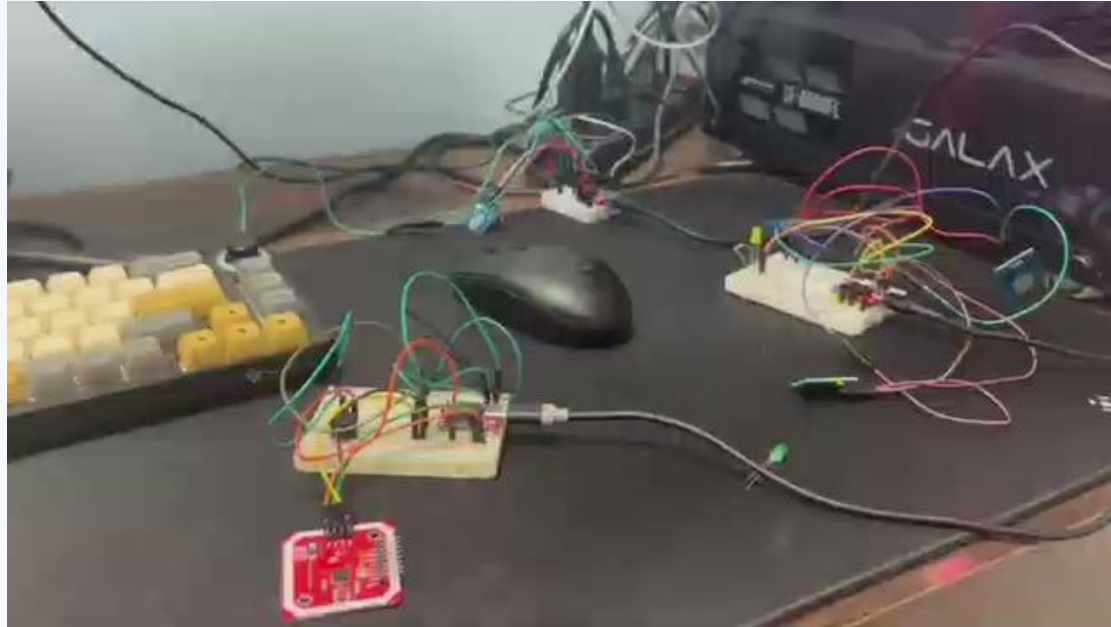
4.2. Principais Limitações

- **Sensibilidade do sensor de toque** (corrigido pelo consenso e pelo timing de atualização do estado dos assentos)
- Apenas um leitor de NFC, já que em um cenário real, um trem teria cerca de 10 portas, e deveria ter 10 sensores por vagão. Além disso, ter 2 leitores NFC criaria uma condição de corrida que não tratamos no projeto
- Distância de conexão dos ESPs (Corrigido pelo servidor central)

4.3. Projetos futuros

- Interação de múltiplos leitores NFC
- Mensagens mais amigáveis por meio de um alto falante (que possivelmente seria integrado no sistema sonoro do trem)
- Diferentes origens do NFC (Celulares, chaveiros, colares)
- Detecção a maior distância
- Confirmação visual da leitura e de disponibilidade no dispositivo NFC
- Sistema de controle de qual pessoa passou o dispositivo NFC e sentou onde, para alertar pessoas que não precisam do assento preferencial

4.4. Vídeo Exemplo:



Obrigad@!

Aline Crispim de Moraes - 14567051

Daniela Yumi Takara - 13659997

Rafaella S. Caminaga - 14108228

Wesley G. R. Fernandes - 13716893

-

CREDITS: This presentation template was created by Slidesgo, and includes icons, infographics & images by Freepik

Além disso, também foram usados como referência os códigos de leitura NFC do repositório da disciplina
